

Received 14 August 2024, accepted 17 October 2024, date of publication 25 October 2024, date of current version 29 November 2024.

Digital Object Identifier 10.1109/ACCESS.2024.3486602

RESEARCH ARTICLE

Enhancing Autonomous Driving With Spatial Memory and Attention in Reinforcement Learning

MATVEY GERASYOV¹, ANDREY V. SAVCHENKO^{2,3},
AND ILYA MAKAROV^{4,5,6}, (Member, IEEE)

¹School of Data Analysis and Artificial Intelligence, HSE University, 101000 Moscow, Russia

²Sber AI Lab, 117312 Moscow, Russia

³Laboratory of Algorithms and Technologies for Network Analysis, HSE University, 603155 Nizhny Novgorod, Russia

⁴Artificial Intelligence Research Institute (AIRI), 105064 Moscow, Russia

⁵ISP RAS, 109004 Moscow, Russia

⁶Artificial Intelligence Research Center, National Research Nuclear University MEPhI, 115409 Moscow, Russia

Corresponding author: Ilya Makarov (iamakarov@misis.ru)

The work of Ilya Makarov was supported by the grant for research centers in the field of AI provided by the Analytical Center (ACRF) in accordance with the agreement on the provision of subsidies (identifier of the agreement 000000D730324P540002) and the agreement with National Research Nuclear University MEPhI No. 70-2023-001309.

ABSTRACT Reinforcement learning in environments with visual observations presents challenges due to incomplete individual observations. The lack of complete information leads to increased uncertainty in decision-making, which requires agents to be supplemented with a memory module to retain information about previous observations. Our paper proposes a novel spatial memory mechanism with a flexible access system based on the multihead attention mechanism. Through experiments in the Atari benchmark and multiple autonomous driving environments, our approach outperforms agents using classical convolutional and recurrent neural networks. Further analysis reveals repeated interpretive patterns in attention distribution among trained agents. This study highlights the effectiveness of spatial memory and attention mechanisms in improving the efficiency of deep reinforcement learning in partially observable environments.

INDEX TERMS Attention, deep reinforcement learning, partially observable Markov decision process.

I. INTRODUCTION

Reinforcement learning focuses on training artificial agents within interactive environments. Through this interaction, the agent receives feedback via scalar rewards, indicating positive or negative behavior and guiding the agent toward its goals. At each step, the agent observes the current state of the environment and chooses an action, after which the environment changes its state and provides new feedback until reaching a terminal state. The agent learns a strategy that maximizes the expected cumulative reward. It is common for the agent to have limited observability of the environment, relying on partial observations, like visuals.

This study considers reinforcement learning in partially observable environments with visual inputs, such as playing video games using only screen pixel values [1], [2], [3], [4] or

robot navigation via camera images [5], [6], [7], [8] or Lidar sensor data [9], [10]. Learning in an environment with visual input includes several challenges. Firstly, the observation space has a large dimensionality. Second, an individual observation often does not contain complete information about the state of the environment. Thus, to learn an informative latent representation of the observation, it is necessary to consider both the visual and the sequential nature of the data at hand.

Deep reinforcement learning commonly uses a combination of a convolutional encoder to reduce the dimensionality of the input image and additional memory to encode the dynamics of agents' trajectories. This integrated approach has shown remarkable improvements in multiple tasks like 3D navigation [11], [12], [13], Atari games [1], [14], and the game of Doom [15], [16], [17], [18], [19]. Memory mechanisms can be either embedded within the agent's network architecture [1], [11], [20] or exist externally [12],

The associate editor coordinating the review of this manuscript and approving it for publication was Claudio Loconsole¹.

[21], [22]. In the embedded case, one applies a recurrent neural network [1], [20] or a transformer [11] to a sequence of vectorized observation representations. The disadvantage of this approach is that the memory module does not store spatial information about the agent's trajectories. Conversely, strategies with external spatial memory [21], [22] are more complex and challenging to scale compared to modules of directly embedded memory, offering improved trajectory information retention, but at the cost of increased complexity.

We propose a new encoder of visual observations featuring an embedded spatial memory that can simultaneously store both spatial and temporal information. In addition, we develop a flexible memory access framework based on the multi-head attention mechanism that allows the agent to query information about the current observation or memory state depending on the task at hand. We demonstrate that our encoder improves over typical feedforward and recurrent baselines on several games from the Atari benchmark [23]. In addition, we test our model in multiple autonomous driving scenarios from *highway-env* [24]. Finally, we perform a saliency analysis of our model and find that its attention heads learn interpretable attention patterns. Furthermore, our findings illustrate the distinct contributions of different attention heads to the agent's policy and value function.

II. BACKGROUND

A. DEEP REINFORCEMENT LEARNING OVERVIEW

The Markov Decision Process (MDP) model is typically used to represent the interaction between the agent and the environment. An MDP is defined by its state and action spaces, a state transition function, and a scalar reward function. A fundamental property of MDP is that the conditional probability distribution given by its state transition function depends only on the current state-action pair s_t, a_t , and is independent of the history of the process. In other words, the Markov decision process is memoryless.

Discounted return is the sum of all rewards starting from state s_t multiplied by a discount factor $\gamma \in [0, 1]$:

$$G_t = \sum_{k=0}^{\infty} \gamma^k r_{t+k+1}, \quad (1)$$

where r_{t+k+1} is the reward received at timestep $t + k + 1$.

The agent makes decisions following its policy $\pi(a|s)$. Reinforcement learning tries to find an optimal policy π^* that maximizes the expected discounted return:

$$\pi^* = \arg \max_{\pi} \mathbb{E}_{\pi} [G_0] \quad (2)$$

The state value function $V_{\pi}(s)$ is the expected discounted return conditional on the state of the environment:

$$V_{\pi}(s) = \sum_a \pi(a|s) \sum_{r,s'} p(r, s' | s, a) [r + \gamma V_{\pi}(s')] \quad (3)$$

where s' is the state following state s . Reward and action at the current step are denoted by r and a , respectively. The value function $v_{\pi}(s)$ represents the value of the following policy π

from state s . Similarly to the state value function, one can introduce the notion of the action value function Q , which is the expected return conditioned on state and action.

Policy-based deep reinforcement learning methods explicitly learn the optimal policy based on the policy gradient theorem [25]:

$$\nabla_{\theta} J = \mathbb{E}_{\pi_{\theta}} [\nabla_{\theta} \log \pi_{\theta}(a|s) Q_{\pi_{\theta}}(s, a)] \quad (4)$$

To reduce the variance of the estimate of (4), one can subtract a baseline function $b(s)$ from the policy gradient. Choosing $b(s)$ to be the state value function $V_{\pi_{\theta}}(s)$ leads to the advantage actor-critic algorithm [26]. The underlying neural network has two heads, actor and critic, representing the policy and state value function, respectively. Equation (4) can be written as follows to obtain the gradient of the actor's objective:

$$\nabla_{\theta} J_{actor} = \frac{1}{N} \sum_{i=0}^N \sum_{s,a} \nabla_{\theta} \log \pi_{\theta}(a|s) A(s, a) \quad (5)$$

where θ denotes the parameters of the network, N is the number of steps in the trajectory, and $A(s, a) = Q(s, a) - V(s)$ is the advantage function. Thus, the advantage function can be interpreted as the benefit of choosing action a in state s compared to the expected baseline, given by the value function. Performing gradient ascend with the policy gradient estimate given by (5), we aim to increase the probability of choosing beneficial actions with positive advantage values. To do so, one can estimate the advantage from a part of a trajectory (s, a, r, s') as follows:

$$\hat{A}(s, a) = r + \gamma V(s') - V(s) \quad (6)$$

Training the critic can be formulated as a regression problem with the following objective:

$$L_{critic} = \frac{1}{N} \sum_{i=0}^N \sum_{s,a} (V_{\theta}(s) - [r + \gamma V(s')])^2 \quad (7)$$

Proximal Policy Optimization (PPO) algorithm [27] additionally constraints the amount of change in the policy by using the following clipped objective function:

$$\hat{\mathbb{E}}_t [\min(r_t(\theta) \hat{A}_t, \text{clip}(r_t(\theta), 1 - \epsilon, 1 + \epsilon) \hat{A}_t)] \quad (8)$$

where $r_t(\theta)$ denotes the probability ratio between the new and old policies, and clip stands for clipping this ratio between $1 - \epsilon$ and $1 + \epsilon$.

B. ConvLSTM ARCHITECTURE

ConvLSTM [28] is a generalization of the LSTM architecture for the case of sequences of visual data. All the inputs, cell outputs, hidden states, and gates of ConvLSTM are three-dimensional tensors whose last two dimensions are spatial, allowing ConvLSTM to store spatial information. Thus, all transitions are performed using the convolution

operation instead of matrix multiplication. The following equations give the forward path through a ConvLSTM cell:

$$\begin{aligned} i_t &= \sigma(W_{xi} * \mathcal{X}_t + W_{hi} * \mathcal{H}_{t-1} + W_{ci} \circ \mathcal{C}_{t-1} + b_i) \\ f_t &= \sigma(W_{xf} * \mathcal{X}_t + W_{hf} * \mathcal{H}_{t-1} + W_{cf} \circ \mathcal{C}_{t-1} + b_f) \\ \mathcal{C}_t &= f_t \circ \mathcal{C}_{t-1} + i_t \circ \tanh(W_{xc} * \mathcal{X}_t + W_{hc} * \mathcal{H}_{t-1} + b_c) \\ o_t &= \sigma(W_{xo} * \mathcal{X}_t + W_{ho} * \mathcal{H}_{t-1} + W_{co} \circ \mathcal{C}_t + b_o) \\ \mathcal{H}_t &= o_t \circ \tanh(\mathcal{C}_t) \end{aligned} \quad (9)$$

Here, “*” stands for the convolution operation. Analogously to the original LSTM [29], i_t , f_t , and o_t are the input, forget, and output gates, and $\mathcal{H}_t$, $\mathcal{C}_t$ represent the hidden state of ConvLSTM. However, unlike LSTM, W and b are the weights and biases of the convolutional layer.

III. RELATED WORK

A. REINFORCEMENT LEARNING IN POMDP

The main challenge in learning in POMDP is that an individual observation does not contain complete information about the state of the environment. The dominant approach to solving this problem in deep reinforcement learning is to augment the agent with a memory containing information about the agent’s trajectory. One of the first examples of such a memory was demonstrated in [30]. The authors proposed the so-called temporal convolution memory; the agent receives the previous k observations as input instead of only the last observation. In the original paper, the authors set k to be equal to four, which later became the standard value for the number of stacked frames and was used unchanged in a large number of subsequent works [31], [32], [33], [34].

A more advanced memory version, first proposed in [20], was the addition of RNN to the agent’s network. An LSTM [29] layer described in Section II-B was applied to a vectorized representation of the observation to store information on the previous steps made by the agent. Subsequently, this approach was used in several further works [26], [35] and received multiple modifications, such as spatial attention [36], using the action-observation pair as the LSTM input [37], and prioritized recurrent experience replay [1].

The transformer architecture [38], [39], [40] can be viewed as a more expressive but less stable alternative to LSTM in the same framework as was shown in [11], [41], and [42]. In [41] and [42], the authors had to limit themselves to offline settings and reduce the reinforcement learning problem to a supervised sequence modeling task. In [11], substantial progress was achieved in applying transformers to online active reinforcement learning was achieved due to the replacement of residual connections in transformers with gating layers.

LSTM-based and transformer-based encoders have a fundamental limitation: they do not store spatial information about the agent’s trajectories. Several papers addressed These limitations with external spatial memory modules [21], [22]. The advantages of such memory modules over the abovementioned approaches are the spatial memory structure

and dedicated read and write operators. However, external memory modules are often more complex and challenging to integrate into existing reinforcement learning systems.

B. ATTENTION IN REINFORCEMENT LEARNING

Attention in deep reinforcement learning was initially introduced in [36]. The authors applied spatial attention to embedding current observation with queries conditioned on the memory state of LSTM and achieved some improvement over the baselines without attention. They also compared the hard and soft attention mechanisms and concluded that the latter performed better. This led to the incredible popularity of soft attention in subsequent studies.

Further, in [43], the authors proposed Memory Q-Network, which stored encoded features of input observations in a dedicated memory and used soft attention conditioned on the current observation to read information from memory.

In [44], the authors applied parallel attention with multiple attention layers to an observation segmented into a grid of patches. Their architecture used randomly initialized trainable bias vectors as queries for attention. On a high level, this approach is similar to the attention in vision transformers [45], which were later applied to reinforcement learning in [46]. The difference is that vision transformers utilize self-attention between image patches instead of trainable bias query vectors. The application of transformers and self-attention to vectorized representations of observations as tokens was covered in the previous subsection.

Our approach is most similar to the one presented in [47] using ConvLSTM and the attention mechanism. The primary and critical difference is that we do not use an additional LSTM to generate queries; instead, we generate them conditional on the input image. Also, we do not split the ConvLSTM state into the key and value tensors. Instead, we concatenate the embeddings obtained from the convolutional encoder with the current state of ConvLSTM before applying attention to allow the agent to access memory or the current input image contents as required.

IV. PROPOSED APPROACH

We propose a new encoder for visual observations and agent memory based on ConvLSTM and attention mechanism. Our model utilizes a flexible memory access framework that, instead of unconditionally using the current state of the memory, allows the agent to access it as needed, depending on the current observation and how much the agent trusts its memory. This allows the agent to search for different objects in memory or the current observation, depending on its current task. In the following subsection, we provide a descriptive summary of our model. Next, we comprehensively cover the workings of the attention mechanism discussed above.

A. ARCHITECTURE OVERVIEW

Figure 1 shows a diagram of our model’s architecture.

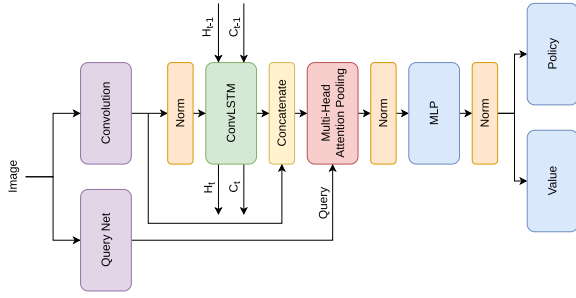


FIGURE 1. Proposed architecture. Norm stands for Layer Normalization. MLP is a block of fully connected layers. Policy and Value correspond to actor and critic heads, respectively. The state of the ConvLSTM memory at time t is denoted by H_t, C_t .

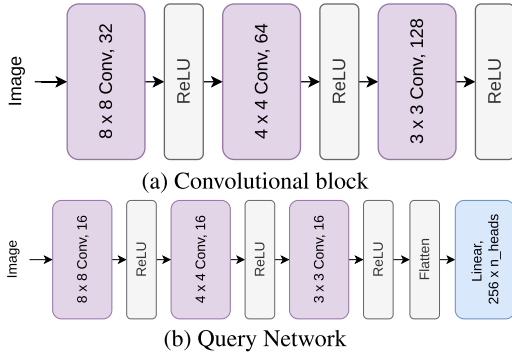


FIGURE 2. Architectures of the convolutional block (a) and Query Network (b). The number of attention heads is denoted by 'n_heads'. The strides in each convolutional layer in both networks are 4, 2, and 1, respectively.

The input images pass through a block of convolutional layers. Its architecture is detailed in Figure 2a. Next, we use ConvLSTM with 128 channels to encode the dynamics of agent trajectories. We choose ConvLSTM because it stores both spatial and temporal information. The new public memory state of ConvLSTM H_t is concatenated with the embedding obtained from the convolutional block, forming a residual connection. Concatenation is performed along the channel axis. This allows us to apply attention to both current observation and memory simultaneously. Attention queries are vectors applied to each spatial location. Each query consists of two parts, corresponding to the visual and temporal components of the data. Thus, the output of each attention head may be affected by the current observation image and accumulated memory to a different degree. This form of spatiotemporal attention makes the agent more adaptive since the optimal trade-off between utilizing the memory and reacting to new information in the current observation may differ for different environments. Finally, the output of all attention heads is aggregated by a block of fully connected layers. In addition, we apply Layer Normalization [48] to the outputs of the convolutional block, the attention pooling layer, and the fully connected block to stabilize the training.

B. ATTENTION MECHANISM

The attention mechanism's query comes from the Query Network (see Figure 2b) conditioned on the input image. An individual query is a vector of size 256. For each spatial location, we compute the dot product of the query vector and the resulting 256-dimensional slice of the input tensor ($X_{i,j} \in \mathbb{R}^{256}$):

$$\begin{aligned} z_{i,j} &= q^T X_{i,j} \\ z_{i,j} &= z_{i,j}^{mem} + z_{i,j}^{obs} \end{aligned} \quad (10)$$

Next, we apply the softmax function to the obtained dot product values to get $height \times width$ attention scores:

$$a_{i',j'} = \frac{\exp(z_{i',j'})}{\sum_{i,j} \exp(z_{i,j})} \quad (11)$$

The attention scores are used as weights to perform global pooling along the spatial dimensions of the input tensor:

$$o = \sum_{i,j} a_{i,j} X_{i,j} \quad (12)$$

This set of operations is performed for each attention head, and the results are concatenated. Thus, for the multi-head case, the output vector o has a dimensionality of $256 \times$ the number of attention heads. Finally, the output vector passes through a block of fully connected layers that outputs the representation used by the actor and critic heads.

V. EXPERIMENT DESIGN AND RESULTS

We compare our model, described in Section IV, with two baselines: a feedforward encoder and a recurrent encoder with LSTM. All three models have the same architecture of the convolutional stack applied to the input image (see Figure 2a). The feedforward baseline uses the last four frames stacked as an input as proposed in [30]. The architecture of the recurrent baseline is analogous to the one presented in [20]. The output of the convolutional block is flattened and goes through an LSTM with 128 units. Our model uses four attention heads throughout all experiments. In the following subsection, we describe hyperparameters and provide details of the training procedure. Next, we state our results. Finally, we perform saliency analysis to determine which parts of the input images affect the output of our model and its individual attention heads.

A. ATARI BENCHMARK

To validate our approach and compare its performance against baselines, we selected three environments from the Atari benchmark [23]: Atlantis, Seaquest, and Kung-FuMaster. Figure 3 shows screenshots from the selected environments.

We train all the agents on 16 environment versions running in parallel. The agent takes a certain fixed number of steps in each environment, and then the collected trajectories are gathered into mini-batches to update the agent's policy

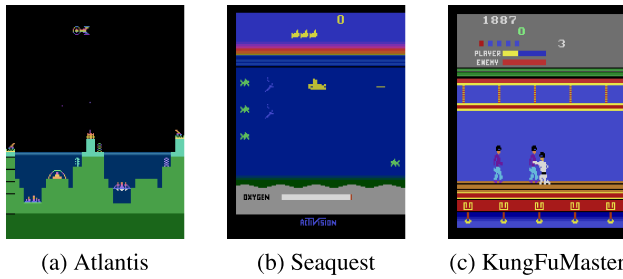


FIGURE 3. Screenshots from the selected Atari environments.

using gradient descent. Training proceeds until a fixed threshold for the total interactions between the agent and the environment is reached. We set the discount factor γ equal to 0.99. We choose the Adam optimizer with a learning rate of 0.001 for gradient optimization. We clip the gradients so that the resulting gradient norm is below 50. This is done to ensure numerical stability of the training process and prevent destructive policy updates. The clipping value was determined by monitoring the gradient norm during parameter tuning.

We use the synchronous advantage actor-critic (A2C) described in Section II-A as the baseline agent. The choice of A2C is motivated by its superior performance and faster convergence speed compared to the vanilla recurrent DQN agent [20]. On the other hand, it offers a simpler alternative relative to subsequent actor-critic agents such as IMPALA [35]. In addition to the policy and value objectives from equations (5) and (7), we apply entropy regularization with a coefficient of 0.005 to prevent premature convergence and facilitate exploration. Agents take 20 steps in each environment per one policy update. All agents are trained for 8 million timesteps total. Observation images are rescaled to a size of 84 by 84 pixels, converted to grayscale, and normalized using running mean and standard deviation. Additionally, we clip all rewards between -1 and 1, similar to [34] and prior work.

Figure 4 shows mean rewards during training. The plots are averaged over three runs with different seeds. As can be seen, our model significantly outperforms both baselines on the games of Atlantis and KungFuMaster and shows faster convergence speed on all three games. Our model and the feedforward baseline reach similar final performances in the game of Seaquest, corresponding to the maximum possible score without air replenishment. To progress further in this game, the agent needs to learn to refill its air reserves to extend its lifetime, which usually requires a lot more frames than considered in this study.

The final human-normalized scores, presented in Table 1, highlight the superiority of our approach in comparison to feedforward and LSTM baselines. It's also important to note that our model with 64 attention heads can run at 30 Hz on a single RTX 4070 GPU during inference. This capability highlights its potential for real-time applications.

TABLE 1. Human-normalized scores on the selected games.

Method	Atlantis	Seaquest	KungFuMaster
A2C FF	693%	8%	137%
A2C LSTM	2001%	4%	165%
A2C attention (ours)	5120%	8%	213%

TABLE 2. Final mean evaluation returns on the selected self-driving scenarios.

Method	merge	roundabout	highway-fast	highway
PPO FF	15.178	9.212	20.714	27.876
PPO LSTM	15.128	9.221	20.477	25.905
PPO attention	15.167	9.246	20.902	29.092

B. AUTONOMOUS DRIVING

In addition to the Atari experiments, we test our approach in four autonomous driving scenarios using environments from *highway-env* [24]. In the *highway* environment, the ego-vehicle is driving on a multilane highway populated with other vehicles. The agent is rewarded for driving at high speed, on the rightmost lanes, while avoiding collisions and staying on the road. In addition, we consider a highway-fast variant that has three lanes instead of four and fewer vehicles in the simulation. In the *merge* environment, the ego-vehicle starts on a main highway and approaches a road junction with incoming vehicles on the access ramp. The objective is maintaining a high speed while allowing cars to merge safely into traffic. The *roundabout* environment simulates a roundabout with flowing traffic, requiring the ego-vehicle to handle lane changes and longitudinal control to pass the roundabout quickly while avoiding collisions. Figure 5 shows screenshots from the respective environments.

The agents take gray-scale images of the environment as input and use *DiscreteMetaAction* action spaces. The available meta-actions consist of changing the target lane and speed and are further used as targets for the low-level controllers. We use the PPO algorithm described in Section II-A with clip fraction ϵ set to 0.1 and the number of epochs set to four. Final evaluation results for all the agents are shown in Table 2. Here, our approach shows comparable or improved performance when contrasted with the baselines.

C. SALIENCY ANALYSIS

To determine which parts of the image affect the decisions of our model, we perform saliency analysis demonstrated in [47] and [49]. The idea of the method is to apply a Gaussian filter at a single pixel and measure the amount of change $S(i, j)$ in the model output. One can form a saliency map S by measuring this change at every pixel in the image. The saliency maps for the agent's policy and value function are defined as $S_{\pi}(i, j) = 0.5\|\pi(X) - \pi(\Phi(X, i, j))\|^2$ and $S_{V_{\pi}}(i, j) = 0.5\|V_{\pi}(X) - V_{\pi}(\Phi(X, i, j))\|^2$ respectively. Here $\Phi(X, i, j)$ denotes the input image perturbed with Gaussian filter at location (i, j) , π are the action probabilities given

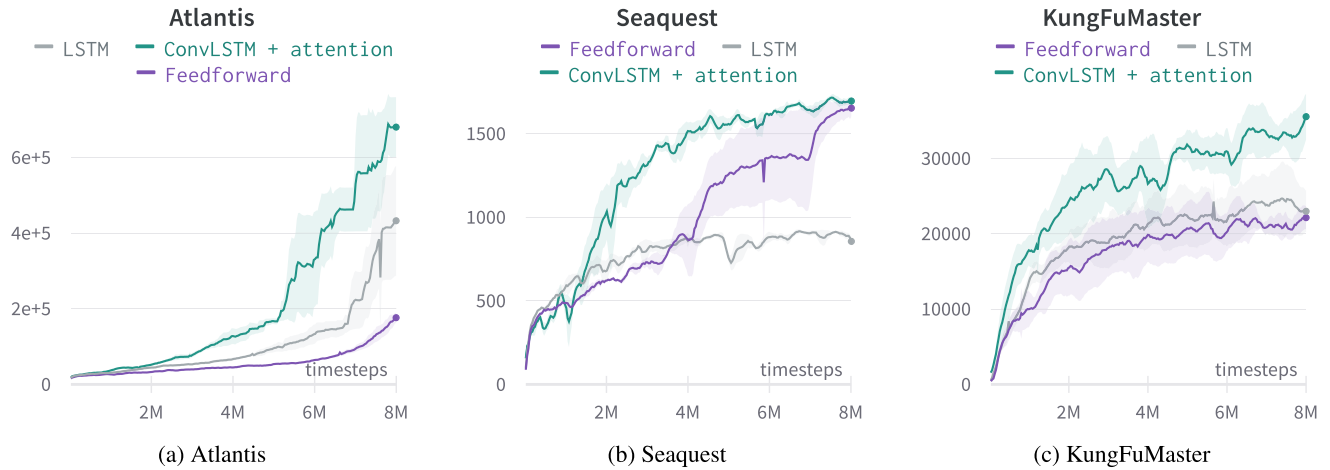


FIGURE 4. Rewards during training on the selected Atari games. The y-axis shows the mean cumulative reward for the last 100 training episodes. The x-axis shows the agent's total number of steps in all environments. The results of our model are shown in green, while the feedforward and LSTM baselines are in purple and gray, respectively.

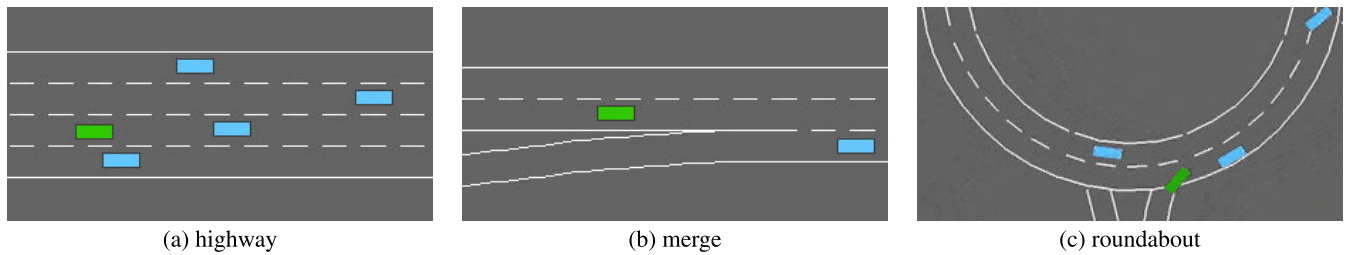


FIGURE 5. Screenshots from the selected autonomous driving environments. The agent is depicted in green.

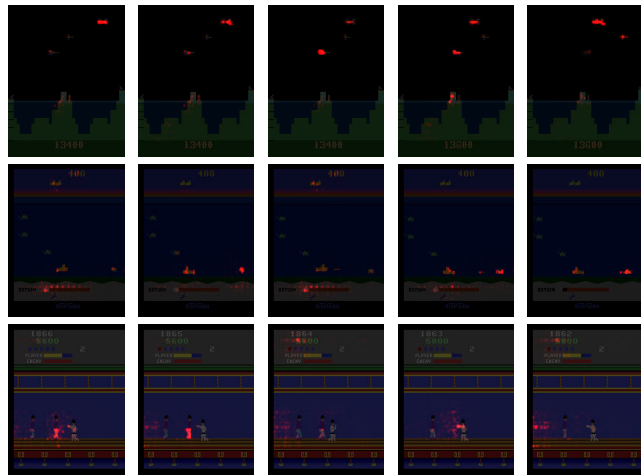


FIGURE 6. Visualization of the policy saliency of our agent on five consecutive frames. Saliency is depicted in red.

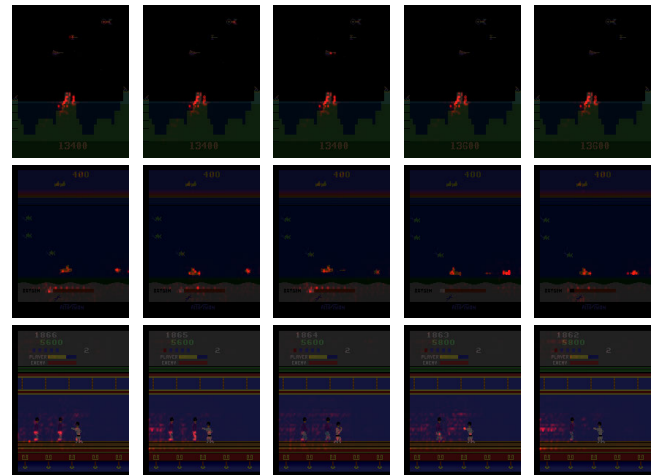


FIGURE 7. Visualization of the value saliency of our agent on five consecutive frames. Saliency is depicted in red.

by the policy, and V_{π} is the value function. Examples of the obtained saliency maps for the policy and value function are shown in Figure 6 and Figure 7, respectively.

Our agent's policy and value saliency focus on objects important to the gameplay, such as the player, enemies, projectiles, and certain parts of the game interface. Policy

saliency is sharper and often corresponds to the agent's target (e.g., the nearest enemy). Value saliency, on the other hand, is more scattered and varies more across games. In the game of Atlantis, the value function focuses almost exclusively on the player's sprite. In the game of KungFuMaster, however, the value saliency most often falls on the enemies and their

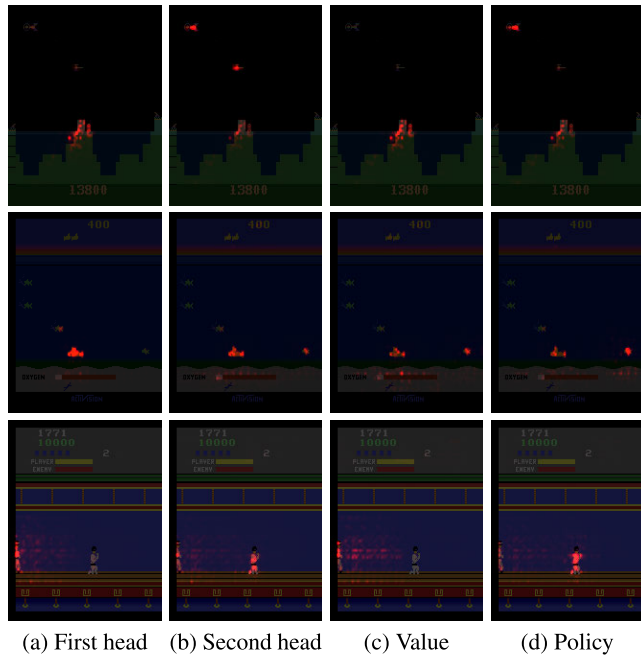


FIGURE 8. Visualization of the saliency of attention heads. The saliency of two out of four attention heads is shown in (a) and (b). Value saliency and policy saliency are shown in (c) and (d) respectively. Saliency is depicted in red.

possible places of appearance. In Seaquest, the value saliency is distributed between the player and the enemies, and the bar displays the amount of air left to the player. We also find that our agent's policy is unexpectedly sensitive to changes in the scores shown in the game interface. We hypothesize that the actor in our model learns to recognize these changes as a signal of completion of its current subtask. Finally, it is worth noting that in the game KungFuMaster, the actor's attention is sufficiently sharp to highlight individual parts of the player and enemy sprites, such as legs or arms during movement (see the third row in Figure 6).

In addition to the policy and value function, we also perform saliency analysis of the attention heads of our model (see Figure 8). We measure the impact that the distortion of the input image at each location produces on the output of individual attention heads. The rest of the analysis goes as outlined above.

We observe that amidst attention heads, some are very player-centric, tracking the player's position and state at every timestep. Others are responsible for locating enemies on the screen or retrieving information from the game interface. We find this separation of roles between attention heads to be consistent across all games. Also, the individual heads contribute distinctly differently to the final policy and value saliency. Another observed pattern is that while individual attention heads may find all enemies on the screen, the policy almost always focuses on the targeted enemy (see the first row in Figure 8). This indicates that our agent properly aggregates and filters information from different attention heads.

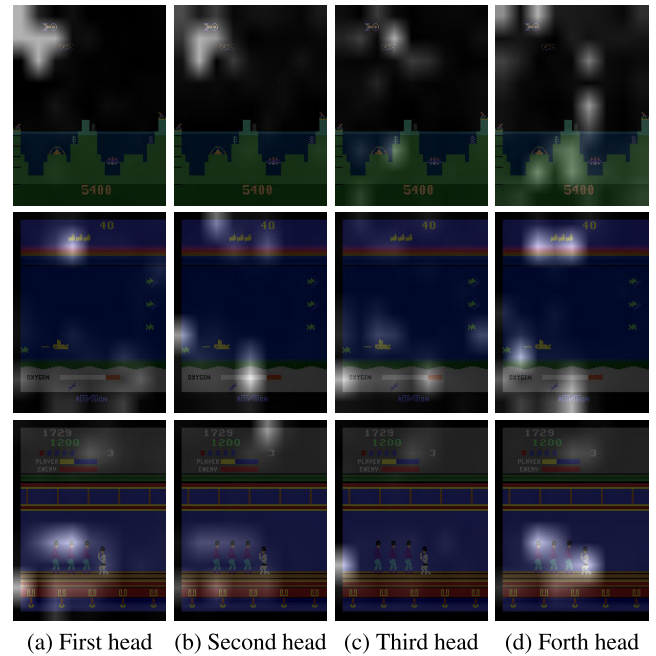


FIGURE 9. Visualization of the contribution of the memory to the final attention score. Regions where memory content has a significant influence on the distribution of attention are highlighted brightly.

D. MEMORY UTILIZATION

Finally, we visualize the contribution of the memory to the final attention score for each spatial position (see Figure 9). This analysis sheds light on the type of information that the agent predominantly retrieves from its memory. We obtain memory contribution by calculating the absolute value of the ratio between the scalar product of the query vector and the memory section and the scalar product of the query vector and the corresponding image encoding section. In other words, the contribution of the memory for each spatial position is computed as $|\frac{z_{mem}}{z_{obs}}|$. Our visualization highlights the agent's selective utilization of memory information, primarily focusing on moving objects and critical aspects of the game interface. Thus, we can conclude that the proposed memory architecture helps the agent retain information about previous observations in spatial locations that are important for its current task.

VI. CONCLUSION

In this study, we proposed a new observation encoder with a spatial memory structure and a flexible attention mechanism. Our model has an increased memory capacity compared to classical recurrent encoders due to the use of ConvLSTM and the ability to retrieve information relevant to the agent's current task from the memory or observation. These advances allowed our model to improve over baselines on several Atari environments with visual observations and self-driving scenarios from *highway-env*. Consequently, our results showcase that the proposed architecture is compatible with different policy learning algorithms. Furthermore, we performed a

saliency analysis of the attention heads of our model and obtained interpretable results. Namely, individual attention heads of our encoder learn distinct roles and have different contributions to the decisions made by the agent. Future work aiming to enhance our model may focus on stabilizing the training process and refining the attention computation algorithm to reduce the training time. Further investigation into the interpretation of the learned attention mechanisms will also be valuable. Finally, while this paper addresses common urban driving scenarios, future research will benefit from experiments in more complex urban environments.

ACKNOWLEDGMENT

(Matvey Gerasyov and Ilya Makarov are co-first authors.)

REFERENCES

- [1] S. Kapturowski, G. Ostrovski, W. Dabney, J. Quan, and R. Munos, "Recurrent experience replay in distributed reinforcement learning," in *Proc. Int. Conf. Learn. Represent. (ICLR)*, 2019, pp. 1–19. [Online]. Available: <https://openreview.net/forum?id=r1lyTjAqYX>
- [2] J. Schrittwieser, T. Hubert, A. Mandhane, M. Barekatin, I. Antonoglou, and D. Silver, "Online and offline reinforcement learning by planning with a learned model," in *Proc. Neural Inf. Process. Syst.*, 2021, pp. 1–12. [Online]. Available: <https://api.semanticscholar.org/CorpusID:233219551>
- [3] A. Zakharenkov and I. Makarov, "Deep reinforcement learning with DQN vs. PPO in VizDoom," in *Proc. IEEE 21st Int. Symp. Comput. Intell. Informat. (CINTI)*. New York, NY, USA: IEEE, Nov. 2021, pp. 000131–000136.
- [4] I. Makarov, A. Kashin, and A. Korinevskaya, "Learning to play pong video game via deep reinforcement learning: Tweaking deep Q-networks versus episodic control," in *Proc. 6th Int. Conf. Anal. Images, Social Netw. Texts (AIST)*. Cham, Switzerland: Polytechnic University, Jul. 2017, pp. 236–241.
- [5] H. Surmann, C. Jestel, R. Marchel, F. Musberg, H. Elhadji, and M. Ardani, "Deep reinforcement learning for real autonomous mobile robot navigation in indoor environments," 2020, *arXiv:2005.13857*.
- [6] K. Kelchtermans and T. Tuytelaars, "RARA: Zero-shot Sim2Real visual navigation with following foreground cues," in *Proc. IEEE/RSJ Int. Conf. Intell. Robots Syst. (IROS)*, Oct. 2022, pp. 1704–1710.
- [7] P. Marza, L. Matignon, O. Simonin, and C. Wolf, "Teaching agents how to map: Spatial reasoning for multi-object navigation," in *Proc. IEEE/RSJ Int. Conf. Intell. Robots Syst. (IROS)*, Oct. 2022, pp. 1725–1732.
- [8] N. Yokoyama, Q. Luo, D. Batra, and S. Ha, "Benchmarking augmentation methods for learning robust navigation agents: The winning entry of the 2021 iGibson challenge," in *Proc. IEEE/RSJ Int. Conf. Intell. Robots Syst. (IROS)*, Oct. 2022, pp. 1748–1755.
- [9] Y. Du, J. Zhang, J. Xu, X. Cheng, and S. Cui, "Global map assisted multi-agent collision avoidance via deep reinforcement learning around complex obstacles," in *Proc. IEEE/RSJ Int. Conf. Intell. Robots Syst. (IROS)*, Oct. 2023, pp. 298–305.
- [10] M. Hutsebaut-Buyse, F. G. Guinjoan, E. Rademakers, S. Latré, A. B. Temsamani, K. Mets, E. Mannens, and T. De Schepper, "Directed real-world learned exploration," in *Proc. IEEE/RSJ Int. Conf. Intell. Robots Syst. (IROS)*, vol. 10, Oct. 2023, pp. 5227–5234.
- [11] E. Parisotto, F. Song, J. Rae, R. Pascanu, C. Gulcehre, S. Jayakumar, M. Jaderberg, R. L. Kaufman, A. Clark, and S. Noury, "Stabilizing transformers for reinforcement learning," in *Proc. Int. Conf. Mach. Learn. (ICML)*, 2020, pp. 7487–7498.
- [12] G. Wayne, C.-C. Hung, D. Amos, M. Mirza, A. Ahuja, A. Grabska-Barwinska, J. Rae, P. Mirowski, J. Z. Leibo, A. Santoro, M. Gemici, M. Reynolds, T. Harley, J. Abramson, S. Mohamed, D. Rezende, D. Saxton, A. Cain, C. Hillier, D. Silver, K. Kavukcuoglu, M. Botvinick, D. Hassabis, and T. Lillicrap, "Unsupervised predictive memory in a goal-directed agent," 2018, *arXiv:1803.10760*.
- [13] I. Makarov and P. Polyakov, "Smoothing Voronoi-based path with minimized length and visibility using composite bezier curves," in *Proc. 5th Int. Conf. Anal. Images, Social Netw. Texts (AIST)*. Cham, Switzerland: Ural Federal University, Apr. 2016, pp. 191–202.
- [14] J. Fan, C. Xiao, and Y. Huang, "GDI: Rethinking what makes reinforcement learning different from supervised learning," 2022, *arXiv:2106.06232*.
- [15] M. Wydmuch, M. Kempka, and W. Jaśkowski, "VizDoom competitions: Playing doom from pixels," *IEEE Trans. Games*, vol. 11, no. 3, pp. 248–259, Jul. 2018.
- [16] I. Makarov, P. Zyuzin, P. Polyakov, M. Tokmakov, O. Gerasimova, I. Guschenko-Cheverda, and M. Uriev, "Modelling human-like behavior through reward-based approach in a first-person shooter game," in *Proc. 3rd Workshop Experim. Econ. Mach. Learn. (EEML)*. Cham, Switzerland: National Research University Higher School of Economics, Jun. 2016, pp. 24–33.
- [17] M. Bakhanova and I. Makarov, "Deep reinforcement learning in VizDoom via DQN and actor-critic agents," in *Proc. 16th Int. Work-Confer. Artif. Neural Netw. (IWANN)*. Berlin, Germany: Springer, Jun. 2021, pp. 138–150.
- [18] V. Sopov and I. Makarov, "Transformer-based deep reinforcement learning in VizDoom," in *Recent Trends in Analysis of Images, Social Networks and Texts*, E. Burnaev, D. I. Ignatov, S. Ivanov, M. Khachay, O. Koltsova, A. Kutuzov, S. O. Kuznetsov, N. Loukachevitch, A. Napoli, A. Panchenko, P. M. Pardalos, J. Saramäki, A. V. Savchenko, E. Tsybalov, and E. Tutubalina, Eds., Cham, Switzerland: Springer, 2022, pp. 96–110.
- [19] A. Sayapin and I. Makarov, "Multi-modal deep reinforcement learning in VizDoom with audio component," in *Proc. IEEE 20th Jubilee Int. Symp. Intell. Syst. Informat. (SISY)*, Sep. 2022, pp. 000437–000442.
- [20] M. Hausknecht and P. Stone, "Deep recurrent Q-learning for partially observable MDPs," 2015, *arXiv:1507.06527*.
- [21] E. Parisotto and R. Salakhutdinov, "Neural map: Structured memory for deep reinforcement learning," 2017, *arXiv:1702.08360*.
- [22] E. Beeching, J. Dibangoye, O. Simonin, and C. Wolf, "EgoMap: Projective mapping and structured egocentric memory for deep RL," in *Machine Learning and Knowledge Discovery in Databases*, F. Hutter, K. Kersting, J. Lijffijt, and I. Valera, Eds., Cham, Switzerland: Springer, 2020, pp. 525–540.
- [23] M. G. Bellemare, Y. Naddaf, J. Veness, and M. Bowling, "The arcade learning environment: An evaluation platform for general agents," *J. Artif. Intell. Res.*, vol. 47, pp. 253–279, Jun. 2013.
- [24] E. Leurent. (2018). *An Environment for Autonomous Driving Decision-making*. [Online]. Available: <https://github.com/eleurent/highway-env>
- [25] R. S. Sutton, D. McAllester, S. Singh, and Y. Mansour, "Policy gradient methods for reinforcement learning with function approximation," in *Proc. 12th Int. Conf. Neural Inf. Process. Syst.* Cambridge, MA, USA: MIT Press, 1999, pp. 1057–1063.
- [26] V. Mnih, A. P. Badia, M. Mirza, A. Graves, T. Lillicrap, T. Harley, D. Silver, and K. Kavukcuoglu, "Asynchronous methods for deep reinforcement learning," in *Proc. Int. Conf. Mach. Learn. (ICML)*, M. F. Balcan and K. Q. Weinberger, Eds., vol. 48. New York, NY, USA: PMLR, Jun. 2016, pp. 1928–1937. [Online]. Available: <https://proceedings.mlr.press/v48/mnih16.html>
- [27] J. Schulman, F. Wolski, P. Dhariwal, A. Radford, and O. Klimov, "Proximal policy optimization algorithms," 2017, *arXiv:1707.06347*.
- [28] X. Shi, Z. Chen, H. Wang, D.-Y. Yeung, W.-K. Wong, and W.-C. Woo, "Convolutional LSTM network: A machine learning approach for precipitation nowcasting," in *Proc. Adv. Neural Inf. Process. Syst.*, vol. 28, 2015, pp. 1–9.
- [29] S. Hochreiter and J. Schmidhuber, "Long short-term memory," *Neural Comput.*, vol. 9, no. 8, pp. 1735–1780, Nov. 1997.
- [30] V. Mnih, K. Kavukcuoglu, D. Silver, A. Graves, I. Antonoglou, D. Wierstra, and M. Riedmiller, "Playing Atari with deep reinforcement learning," 2013, *arXiv:1312.5602*.
- [31] H. Van Hasselt, A. Guez, and D. Silver, "Deep reinforcement learning with double Q-learning," in *Proc. AAAI Conf. Artif. Intell.*, 2016, vol. 30, no. 1, pp. 2094–2100.
- [32] Z. Wang, T. Schaul, M. Hessel, H. Hasselt, M. Lanctot, and N. Freitas, "Dueling network architectures for deep reinforcement learning," in *Proc. Int. Conf. Mach. Learn. (ICML)*, 2016, pp. 1995–2003.
- [33] T. Schaul, J. Quan, I. Antonoglou, and D. Silver, "Prioritized experience replay," 2015, *arXiv:1511.05952*.
- [34] M. Hessel, J. Modayil, H. Van Hasselt, T. Schaul, G. Ostrovski, W. Dabney, D. Horgan, B. Piot, M. Azar, and D. Silver, "Rainbow: Combining improvements in deep reinforcement learning," in *Proc. AAAI Conf. Artif. Intell.*, 2018, vol. 32, no. 1, pp. 3215–3222.

- [35] L. Espeholt, H. Soyer, R. Munos, K. Simonyan, V. Mnih, T. Ward, Y. Doron, V. Firoiu, T. Harley, and I. Dunning, "Impala: Scalable distributed deep-RL with importance weighted actor-learner architectures," in *Proc. Int. Conf. Mach. Learn. (ICML)*, 2018, pp. 1407–1416.
- [36] I. Sorokin, A. Seleznev, M. Pavlov, A. Fedorov, and A. Ignateva, "Deep attention recurrent Q-network," 2015, *arXiv:1512.01693*.
- [37] P. Zhu, X. Li, P. Poupart, and G. Miao, "On improving deep reinforcement learning for POMDPs," 2017, *arXiv:1704.07978*.
- [38] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, L. U. Kaiser, and I. Polosukhin, "Attention is all you need," in *Advances in Neural Information Processing Systems*, I. Guyon, U. V. Luxburg, S. Bengio, H. Wallach, R. Fergus, S. Vishwanathan, and R. Garnett, Eds., vol. 30. Red Hook, NY, USA: Curran Associates, 2017.
- [39] T. Kolmakova and I. Makarov, "Efficient transformer for video summarization," in *Proc. Adv. Comput. Intell.*, Oct. 2023, pp. 52–65.
- [40] A. Luginov and I. Makarov, "SwiftDepth: An efficient hybrid CNN-transformer model for self-supervised monocular depth estimation on mobile devices," in *Proc. IEEE Int. Symp. Mixed Augmented Reality Adjunct (ISMAR-Adjunct)*, Oct. 2023, pp. 642–647.
- [41] L. Chen, K. Lu, A. Rajeswaran, K. Lee, A. Grover, M. Laskin, P. Abbeel, A. Srinivas, and I. Mordatch, "Decision transformer: Reinforcement learning via sequence modeling," in *Proc. Adv. Neural Inf. Process. Syst.*, vol. 34, 2021, pp. 15084–15097.
- [42] M. Janner, Q. Li, and S. Levine, "Offline reinforcement learning as one big sequence modeling problem," in *Proc. Adv. Neural Inf. Process. Syst.*, vol. 34, 2021, pp. 1273–1286.
- [43] J. Oh, V. Chockalingam, and H. Lee, "Control of memory, active perception, and action in Minecraft," in *Proc. Int. Conf. Mach. Learn.*, 2016, pp. 2790–2799.
- [44] J. Choi, B.-J. Lee, and B.-T. Zhang, "Multi-focus attention network for efficient deep reinforcement learning," in *Proc. Workshops 31st AAAI Conf. Artif. Intell.*, 2017, pp. 1–7.
- [45] A. Dosovitskiy, L. Beyer, A. Kolesnikov, D. Weissenborn, X. Zhai, T. Unterthiner, M. Dehghani, M. Minderer, G. Heigold, S. Gelly, J. Uszkoreit, and N. Houlsby, "An image is worth 16×16 words: Transformers for image recognition at scale," in *Proc. ICLR*, 2021, pp. 1–21.
- [46] E. Kargar and V. Kyrki, "Vision transformer for learning driving policies in complex multi-agent environments," 2021, *arXiv:2109.06514*.
- [47] A. Mott, D. Zoran, M. Chrzanowski, D. Wierstra, and D. Jimenez Rezende, "Towards interpretable reinforcement learning using attention augmented agents," in *Proc. Adv. Neural Inf. Process. Syst.*, vol. 32, 2019, pp. 1–10.
- [48] J. L. Ba, J. Ryan Kiros, and G. E. Hinton, "Layer normalization," 2016, *arXiv:1607.06450*.
- [49] S. S. Guo, R. Zhang, B. Liu, Y. Zhu, D. Ballard, M. Hayhoe, and P. Stone, "Machine versus human attention in deep reinforcement learning tasks," in *Proc. Adv. Neural Inf. Process. Syst.*, vol. 34, 2021, pp. 25370–25385.



MATVEY GERASYOV received the bachelor's degree in computer science from Moscow State Technical University of Civil Aviation, Moscow, Russia, and the master's degree in data science from the National Research University Higher School of Economics, Moscow, in 2022.

He is currently with the School of Data Analysis and Artificial Intelligence, National Research University Higher School of Economics, where he is also continuing his research. His contributions

include model code and experiments and paper writing.



ANDREY V. SAVCHENKO received the B.S. degree in applied mathematics and informatics from Nizhny Novgorod State Technical University, Nizhny Novgorod, Russia, in 2006, the Ph.D. degree in mathematical modeling and computer science from The State University–Higher School of Economics, Moscow, Russia, in 2010, and the Ph.D. degree in system analysis and information processing from Nizhny Novgorod State Technical University, in 2016. Since 2022, he has been with

the Sber AI Lab, where he is currently the Scientific Director. He is also the Leading Research Fellow of the Laboratory of Algorithms and Technologies for Network Analysis, HSE University, Nizhny Novgorod. He has authored or co-authored one monograph and more than 50 articles. His contribution includes paper revision. His research interests include statistical pattern recognition, image classification, and biometrics.



ILYA MAKAROV (Member, IEEE) received the Specialist degree in mathematics from Lomonosov Moscow State University, Moscow, Russia, and the Ph.D. degree in computer science from the University of Ljubljana, Ljubljana, Slovenia.

Since 2011, he has been a Lecturer with the School of Data Analysis and Artificial Intelligence, HSE University, where he was the School Deputy Head, from 2012 to 2016. He was also the Program Director of the BigData Academy

MADE, V. K., and a Researcher with the Samsung-PDMI Joint AI Center, St. Petersburg Department, V. A. Steklov Mathematical Institute, Russian Academy of Sciences, Saint Petersburg, Russia. He became the Head of the AI Center and Data Science Tech Master Program, NLP, National University of Science and Technology MISIS. He is currently an Associate Professor and a Senior Research Fellow. He is a Senior Research Fellow with the Artificial Intelligence Research Institute (AIRI), Moscow, where he leads research in industrial AI. His contributions include paper revision, help with experiment and model design, and research supervision.

...